

Introduction

Doom is a documentation development tool designed for internal use at Alauda, built on top of [rspress](#)[↗]. It provides users with a rich set of built-in plugins for an out-of-the-box experience.

TOC

Core Capabilities

Based on Markdown and its extension MDX

Get Started

Getting Started

Usage

APIs

Core Capabilities

- Automatically generates a configurable weight (order) left sidebar
- Full-text static document search
- Multilingual support

Based on Markdown and its extension MDX

MDX is a powerful content development approach. You can write Markdown files as you normally would while also using React components within the Markdown content:



```
// docs/index.mdx
import { CustomComponent } from './custom';

# Hello World

<CustomComponent />
```

For more details, you can refer to the [“Using MDX” documentation](#).

Get Started

Let's [get started quickly](#) with Doom!

Getting Started

Getting Started

Creating a Project

CLI Tool

Usage

Configuration

Configure the `doom` documentation tool

Convention

Based on the principle of "convention over documents to automatically generate the l

Markdown

Callouts

- Configuration File
- Basic Configuration
- API Documentation Configuration
- Permission Documentation Configuration
- Reference Documentation Configuration
- Release Notes Configuration
- Sidebar Configuration
- Internal Documentation Routes Configuration
- Only Include Documentation Routes Configuration
- Syntax Highlighting Plugin Configuration
- `sites.yaml` Configuration
- Translation Configuration
- Editing Documentation in Code Repository
- Documentation Export Configuration
- Documentation Lint Configuration
- Algolia Search Configuration
- Sitemap Configuration

- Directory Structure
- Metadata
- Sorting
- Preview

- Mermaid
- MDX**
- Using MDX enable
- rspress Component
- doom Component
- Custom Component

Internationalization

Using Internationalized Text in Reusable Component

- `i18n.json`
- `.ts/.tsx`
- `.mdx`

- API Documentation**
- K8S API
- Advanced API
- CRD (deprecated)
- Common Reference
- Specifying operation

Permission Description Document

- `props`
- Example

- Referencing**
- Document Reference

Deployment

- After the documentation project development is completed, we can deploy to the target platform
- Build and Preview
- Multi-version Build
- Merged Directory Structure
- Dynamic Mount Configuration File

APIs

[Advanced APIs](#)

[CRDs](#)

[Kubernetes](#)

[Common References](#)

Getting Started

TOC

Creating a Project

CLI Tool

Starting the Development Server

Production Build

Local Preview

Using Scaffolding Templates

Translating Documentation

Exporting PDF

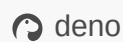
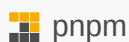
Documentation Linting

Creating a Project

First, you can create a new directory with the following command:

```
mkdir my-docs && cd my-docs
```

Run `npm init -y` to initialize a project. You can install doom using npm, yarn, or pnpm:



```
npm install -D @alauda/doom typescript
```

Then create files with the following commands:

```
# Create docs directories, default supports bilingual Chinese and English
mkdir docs/en && echo '# Hello World' > docs/en/index.md
mkdir docs/zh && echo '# 你好世界' > docs/zh/index.md
```

Add the following scripts to your `package.json`:

```
{
  "scripts": {
    "dev": "doom dev",
    "build": "doom build",
    "new": "doom new",
    "serve": "doom serve",
    "translate": "doom translate",
    "export": "doom export"
  }
}
```

Show more ▾

Then initialize a configuration file `doom.config.yml`:

```
title: My Docs
```

Also create a `tsconfig.json` file with the following content:

```
{
  "compilerOptions": {
    "jsx": "react-jsx",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "resolveJsonModule": true,
  }
}
```

Show more ▾

Finally, create a `global.d.ts` file with the following content:

```
/// <reference types="@alauda/doom/runtime" />
```

This allows you to use doom's global components in `.mdx` files with type safety.

CLI Tool

```
doom -h
```

```
# output
```

```
Usage: doom [options] [command]
```

```
Doctor Doom making docs.
```

```
Options:
```

Show more ▾

For more configuration, please refer to [Configuration](#)

Starting the Development Server

Run `yarn dev` to start the development server, the browser will automatically open the documentation homepage.

```
doom dev -h
```

```
# output
```

```
Usage: doom dev [options] [root]
```

```
Start the development server
```

```
Arguments:
```

Show more ▾

Production Build

Run `yarn build` to build the production code. After building, static files will be generated in the `dist` directory.

Local Preview

Run `yarn serve` to preview the built static files. Note that if you used `-b` or `-p` parameters during build, you need to use the same `-b` and `-p` parameters during preview.

Using Scaffolding Templates

Run `yarn new` to generate projects, modules, or documentation using scaffolding templates.

Translating Documentation

```
doom translate -h
```

```
# output
```

```
Usage: doom translate [options] [root]
```

```
Translate the documentation
```

```
Arguments:
```

Show more ▾

- The `-g, --glob` parameter is required. You can specify the directories or files to translate, supporting `glob` syntax. Note that the parameter value must be quoted to avoid unexpected behavior caused by command line parsing. Examples:
 1. `yarn translate -g abc xyz` will translate all documents under `<root>/<source>/abc` and `<root>/<source>/xyz` to `<root>/<target>/abc` and `<root>/<target>/xyz` respectively.
 2. `yarn translate -g '*'` will translate all documents under `<root>/<source>`.

- The `-C, --copy` parameter is optional. It controls whether to copy local resource files to the target directory when the target file does not exist. The default is `false`, which means changing the resource file reference path to the source path. Examples:
 - When enabled:
 1. When translating `/<source>/abc.jpg`, it will copy `<root>/public/<source>/abc.jpg` to `<root>/public/<target>/abc.jpg` and modify the reference path in the document to `/<target>/abc.jpg`.
 2. In `<root>/<source>/abc.mdx`, when translating `./assets/xyz.jpg`, it will copy `<root>/<source>/assets/xyz.jpg` to `<root>/<target>/assets/xyz.jpg`, and the image reference path remains unchanged.
 3. In `<root>/<source>/abc.mdx`, when translating `./assets/<source>/xyz.jpg`, it will copy `<root>/<source>/assets/<source>/xyz.jpg` to `<root>/<target>/assets/<target>/xyz.jpg` and modify the reference path in the document to `./assets/<target>/xyz.jpg`.
 - When not enabled:
 1. When translating `/<source>/abc.jpg`, if `<root>/public/<target>/abc.jpg` exists, the reference path in the document will be changed to `/<target>/abc.jpg`; otherwise, the image reference path remains unchanged.
 2. In `<root>/<source>/abc.mdx`, when translating `./assets/<source>/xyz.jpg`, if `<root>/<target>/assets/<target>/xyz.jpg` exists, the reference path will be changed to `./assets/<target>/xyz.jpg`; otherwise, it will be changed to `../<source>/assets/<target>/xyz.jpg`.

WARNING

Specially, if you use `-g '*'` for full translation, the file lists of `source` and `target` directories will be compared, and unmatched `target` files except for `internalRoutes` will be automatically deleted.

TIP

The translation feature requires configuring the `AZURE_OPENAI_API_KEY` environment variable locally. Please contact your team leader to obtain it.

You can control translation behavior in the document metadata:

```
i18n:
  title:
    en: DevOps Connectors
  additionalPrompts: 'The term Connectors in this document is a proper no
un and should not be translated'
  disableAutoTranslation: false
title: DevOps Connectors
```

For more configuration, please refer to [Translation Configuration](#)

Exporting PDF

WARNING

Please run the `yarn build` command before exporting.

```
doom export -h
```

```
# output
```

```
Usage: doom export [options] [root]
```

```
Export the documentation as PDF, `apis/**` and `*/apis/**` routes will be
ignored automatically
```

Show more ▾

Run `yarn export` to export the documentation as PDF files. Note that if you used `-b` or `-p` parameters during build, you need to use the same `-b` and `-p` parameters during export.

The export feature depends on [playwright](#) [↗]. For CI pipelines, please use [build-harbor.alauda.cn/frontend/playwright-runner:doom](#) as the base image for dependency installation and documentation build. Locally, you can set the following environment variable to speed up downloads:

```
.env.yarn
```

```
PLAYWRIGHT_DOWNLOAD_HOST="https://cdn.npmmirror.com/binaries/playwright"
```

Besides exporting a complete PDF for the entire site, [doom](#) also supports exporting a single PDF file for a specified entry. For more configuration, please refer to [Documentation Export Configuration](#)

Documentation Linting

```
doom lint -h
```

```
# output
```

```
Usage: doom lint [options] [root]
```

```
Lint the documentation
```

```
Arguments:
```

Show more [▼]

[doom lint](#) is based on [ESLint](#) [↗] and [cspell](#) [↗]. For a better experience in your editor, you can install the corresponding plugins [ESLint](#) [↗] / [CSpell](#) [↗], then create the corresponding configuration files:

- [eslint.config.mjs](#):

```
import doom from '@alauda/doom/eslint'

export default doom(new URL('docs', import.meta.url))
```

- [cspell.config.mjs](#):

```
export { default } from '@alauda/doom/cspell'
```

We also agree that the `.cspell` folder under the current working directory (`CWD`) is used to store CSpell dictionary files. For example, you can create `.cspell/k8s.txt` :

```
k8s
kubernetes
```

For more configuration, please refer to [Lint Configuration](#)

Usage

Configuration

Configure the `doom` documentation tool

Configuration File

Basic Configuration

API Documentation Configuration

Permission Documentation Configuration

Reference Documentation Configuration

Release Notes Configuration

Sidebar Configuration

Internal Documentation Routes Configuration

Only Include Documentation Routes Configuration

Syntax Highlighting Plugin Configuration

`sites.yaml` Configuration

Translation Configuration

Editing Documentation in Code Repository

Documentation Export Configuration

Documentation Lint Configuration

Algolia Search Configuration

Sitemap Configuration

Convention

Based on the principle of "convention over configuration" to automatically generate the layout

Directory Structure

Metadata

Sorting

Preview

Markdown

Callouts

Mermaid

MDX

Using MDX enable

reactpress Component

doom Component

Custom Component

Internationalization

Using Internationalized Text in Reusable Components

`i18n.json`

`.ts/.tsx`

`.mdx`

API Documentation

K8S API

Advanced API

CRD (deprecated)

Common Reference

Specifying operation

Permission Description Document

`props`

Example

Referencing

Document Reference

Deployment

After the documentation project development is completed, we can deploy the project to the target platform

Build and Preview

Multi-version Build

Merged Directory Structure

Dynamic Mount Configuration File

Configuration

TOC

[Configuration File](#)

Basic Configuration

API Documentation Configuration

Permission Documentation Configuration

Reference Documentation Configuration

`frontmatterMode`

Release Notes Configuration

Sidebar Configuration

Internal Documentation Routes Configuration

Only Include Documentation Routes Configuration

Syntax Highlighting Plugin Configuration

`sites.yaml` Configuration

Translation Configuration

Editing Documentation in Code Repository

Documentation Export Configuration

Documentation Lint Configuration

Algolia Search Configuration

Sitemap Configuration

Configuration File

In most cases, we only need to use a static `yaml` configuration file, supporting `doom.config.yaml` or `doom.config.yml`. For complex scenarios, such as requiring dynamic configuration or custom `rspress` plugins, `js/ts` configuration files can be used, supporting multiple file formats including `.js/.ts/.mjs/.mts/.cjs/.cts`.

For `js/ts` configuration files, we need to export the configuration, which can be combined with the `defineConfig` function exported from `@alauda/doom/config` to provide type assistance:

```
import { defineConfig } from '@alauda/doom/config'

export default defineConfig({})
```

Basic Configuration

- `lang`: Default documentation language. To accommodate most projects, we support both Chinese and English documents by default. The default language is `en`. If the current documentation project does not require multilingual support, this can be set to `null` or `undefined`.
- `title`: Documentation title, displayed on the browser tab.
- `logo`: Logo at the top left of the documentation, supports image URLs or file paths. Absolute paths refer to files under the `public` directory, relative paths refer to files relative to the current tool directory. By default, the built-in Alauda logo from the `doom` package is used.
- `logoText`: Documentation title, displayed next to the logo at the top left.
- `icon`: Documentation favicon, defaults to the same as `logo`.
- `base`: Base path for the documentation, used when deploying to a non-root path, such as `product-docs`. Defaults to `/`.
- `outDir`: Directory for build outputs, defaults to `dist/{base}/{version}`. If specified, it changes to `dist/{outDir}/{version}`, where `version` is optional. See [Multi-version Build](#) for reference.

API Documentation Configuration

```
api:
  # CRD definition file paths, relative to the directory of doom.config.
  *, supports glob matching, json/yaml files
crds:
  - docs/shared/crds/*.yaml
  # OpenAPI definition file paths, relative to the directory of doom.conf
  ig.*, supports glob matching, json/yaml files
openapis:
```

[Show more](#) ▾

Refer to [API Documentation](#) for writing documentation.

Permission Documentation Configuration

```
# The following resource file paths are relative to the directory of doo
m.config.*, support glob matching, json/yaml files
permission:
  functionresources:
    # `kubectl get functionresources`
    - docs/shared/functionresources/*.yaml
  roletemplates:
    # `kubectl get roletemplates -l auth.cpaas.io/roletemplate.official=t
```

[Show more](#) ▾

Refer to [Permission Documentation](#) for writing documentation.

Reference Documentation Configuration

reference:

- **repo:** `alauda-public/product-doc-guide` # Optional, repository address of the referenced documentation. If not filled, the current documentation repository address is used by default.

branch: # [string] Optional, branch of the referenced documentation repository

publicBase: # [string] Optional, when using a remote repository, the absolute path where static resources corresponding to `/images/xx.png` are

Show more ▾

frontmatterMode

- **ignore**: Ignore the frontmatter of the referenced document, keep using the current document's frontmatter.
- **merge**: Merge the frontmatter of the referenced document. If keys conflict, the referenced document's values override the current document's.
- **replace**: Replace the current document's frontmatter with that of the referenced document.
- **remove**: Remove the current document's frontmatter.

Refer to [Reference Documentation](#) for writing documentation.

Release Notes Configuration

releaseNotes:**queryTemplates:**

fixed: # JQL statements that can include ejb templates

unfixed:

release-notes.md

```
<!-- release-notes-for-bugs?template=fixed&project=DevOps -->
```

```
release-notes.mdx
```

```
{/* release-notes-for-bugs?template=fixed&project=DevOps */}
```

Taking the above `template=fixed&project=DevOps` as an example, `fixed` is the template name defined in `queryTemplates`. The remaining `query` parameter `project=DevOps` is passed as `ejs` [↗](#) template parameters to the `fixed` template, which after processing is used to form a jira `jql` [↗](#) request to `https://jira.alauda.cn/rest/api/2/search?jql=<jql>`. This API requires authentication, and the environment variables `JIRA_USERNAME` and `JIRA_PASSWORD` must be provided to preview the effect.

Sidebar Configuration

```
sidebar:
  collapsed: false # Optional, whether the sidebar is collapsed by default. Defaults to collapsed. When documentation content is small, consider setting to false.
```

Internal Documentation Routes Configuration

```
internalRoutes: # Optional, supports glob matching, relative to the docs directory. When the CLI option `-i, --ignore` is enabled, matched routes/files will be ignored.
  - '*/internal/**'
```

Only Include Documentation Routes Configuration

```

onlyIncludeRoutes: # Optional, supports glob matching, relative to the docs directory. When the CLI option `-i, --ignore` is enabled, only routes/files under this configuration will be enabled. Can be combined with `internalRoutes` to further exclude some routes.
  - '*/internal/**'
internalRoutes:
  - '*/internal/overview.mdx'

```

Syntax Highlighting Plugin Configuration

```

shiki:
  theme: # optional, https://shiki.style/themes
  langs: # optional, https://shiki.style/languages
  transformers: # optional, only available in js/ts config, https://shiki.style/guide/transformers

```

WARNING

Unconfigured languages will trigger warnings in the command line and fall back to rendering as `plaintext`.

`sites.yaml` Configuration

The `sites.yaml` configuration file is used to configure subsite information associated with the current documentation site. This information is used by [External Site Components](#) and when building single-version documentation.

```
- name: connectors # Globally unique name
  base: /devops-connectors # Base path for site access
  version: v1.1 # Version used for ExternalSite/ExternalSiteLink redirection when building multi-version sites

  displayName: # Site display name. If not filled or language not matched, defaults to `name`
  en: DevOps Connectors
```

[Show more ▾](#)

Translation Configuration

```
translate:
  # System prompt, ejs template. Parameters passed in include `sourceLang`, `targetLang`, `userPrompt`, `additionalPrompts`, `terms`, `titleTranslationPrompt`.
  # Among them, `sourceLang` and `targetLang` are the strings `□□` and `□□`,
  # `userPrompt` is the global user configuration below, which may be empty
```

[Show more ▾](#)

Editing Documentation in Code Repository

```
editRepoBaseUrl: alauda/doom/tree/main/docs # The https://github.com/ prefix can be omitted. Only effective when the CLI flag `-R, --edit-repo` is enabled.
```

Documentation Export Configuration

```
export:
  - name: Concepts # Optional, globally unique PDF name, defaults to the
documentation title
  scope: '*/concepts' # Required, string or array, document scope, supports
glob matching, relative to the docs directory
```

Documentation Lint Configuration

```
lint:
  cspellOptions: # Optional, cspell configuration options, refer to https://github.com/streetsidesoftware/cspell/tree/main/packages/cspell-eslint-plugin#options
```

Algolia Search Configuration

```
algolia: # Optional, Algolia search configuration, only effective when the CLI flag `-a, --algolia` is enabled
  appId: # Algolia Application ID
  apiKey: # Algolia API Key
  indexName: # Algolia index name
```

Please use `public/robots.txt` for Algolia crawler verification.

Info

Due to the current architecture limitations of `rspress`, using Algolia search requires implementing via a [custom theme](#). To unify usage of related theme features, we provide the

`@alauda/doom/theme` theme entry. Please add the following theme configuration file to enable:

```
theme/index.ts
```

```
export * from '@alauda/doom/theme'
```

Sitemap Configuration

```
siteUrl: https://docs.alauda.cn # Optional, site URL used to generate sit  
emap, only effective when the CLI flag `-S, --site-url` is enabled
```

Convention

TOC

[Directory Structure](#)[Metadata](#)[Sorting](#)[Preview](#)

Directory Structure

The left sidebar is automatically generated based on the file directory structure, where the `index` file in the first-level directory acts as the document's homepage and will display as the first item in the left navigation. Subfolders can use `index.md` or `index.mdx` and define the first-level title to set the grouping title for the left sidebar. Other sub-documents will be automatically merged into the current group, and nested subfolders will follow the same rules.

```
├─ index.md
├─ start.mdx
└─ usage
    ├─ index.mdx
    └─ convention.md
```

We also agree that:

1. The `public` directory is used to store static resources such as images, videos, etc.

2. The `public/_remotes` directory is used to store static resources associated with [remote reference documents](#). Please do not directly rely on resources from this directory; you may add `*/public/_remotes` to `.gitignore` to prevent these from being committed to the code repository.
3. The `shared` directory is for storing common components, reusable documents, etc., and will not automatically generate document data.

Metadata

At the beginning of the document, you can define the document's metadata such as title, description, author, category, etc., through the `frontmatter`.

```
---  
title: Title  
description: Description  
author: Author  
category: Category  
---
```

In the body of the document, when using `.mdx` files, you can access these metadata through `frontmatter` as described in [MDX](#).

Sorting

Other documents, except for `index.md` or `index.mdx`, will be sorted by default according to their file names. You can customize the `weight` value in the `frontmatter` to adjust the order of documents in the left sidebar (the smaller the `weight` value, the higher the priority in sorting).

```
---  
weight: 1  
---
```

Warning

Note: Currently, changes to the left navigation configuration require a service restart to take effect, and it is usually not necessary to pay too much attention during development.

Preview

Sometimes, we do not need to display special content on the group homepage. In this case, you can use `index.mdx` file and the `Overview` component to display the list of documents in the current group. This will showcase the titles, descriptions, and secondary title information of the grouped list file.

Usage

```
<Overview />
```

You can refer to [Usage](#) for the effect.

Markdown

In addition to the standard [gfm](#) [↗] syntax, Doom includes some additional Markdown extension features.

TOC

Callouts

Mermaid

Callouts

Source code annotation component

Note

1. Please use inline code comments according to the actual language, such as `;`, `%`, `#`, `//`, `/** */`, `--`, and `<!-- -->`, etc.
2. If you want to treat them as code comments, use `[\\!code callout]` for escaping.
3. Sometimes, `:::callouts` may display abnormally due to nested indentation; you can use `<div class="doom-callouts">` or the `<Callouts>` component instead.

```

```sh
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
+ 218 MiB \ # [!code callout]
+ 8 MiB × (number of vCPUs) \ # [!code callout]
+ 16 MiB × (number of graphics devices) \ # [!code callou
t]
+ (additional memory overhead) # [!code callout]
```

```

Show more ▾

```

Memory overhead per virtual machine ≈ (1.002 × requested memory) \
+ 218 MiB \ ①
+ 8 MiB × (number of vCPUs) \ ②
+ 16 MiB × (number of graphics devices) \ ③
+ (additional memory overhead) ④

```

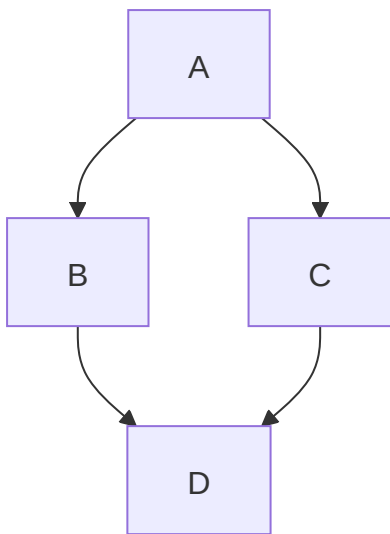
1. Required for the processes that run in the `virt-launcher` pod.
2. Number of virtual CPUs requested by the virtual machine.
3. Number of virtual graphics cards requested by the virtual machine.
4. Additional memory overhead:
 - If your environment includes a Single Root I/O Virtualization (SR-IOV) network device or a Graphics Processing Unit (GPU), allocate 1 GiB additional memory overhead for each device.
 - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
 - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

For more source code transformation features, please refer to [Shiki Transformers](#) ↗.

Mermaid ↗

Chart drawing tool

```
```mermaid
graph TD;
 A-->B;
 A-->C;
 B-->D;
 C-->D;
```
```



With [Markdown Preview Mermaid](#) ↗, you can preview in real-time in VSCode.

MDX

[MDX ↗](#) is an extension syntax of Markdown that allows the use of JSX syntax within Markdown. For usage, refer to [rspress MDX ↗](#).

TOC

[rspress Components](#)

doom Components

[Overview](#)[Directive](#)[ExternalSite](#)[ExternalSiteLink](#)[AcpApisOverview](#) and [ExternalApisOverview](#)

Term

[props](#)[TermsTable](#)[props](#)[JsonViewer](#)

Custom Component Reuse

rspress Components

Most of the [built-in components](#) provided by the `rspress` theme have been adjusted to global components, which can be used directly in `.mdx` files without import, including:

- `Badge`
- `Card`
- `LinkCard`
- `PackageManagerTabs`
- `Steps`
- `Tab/Tabs`
- `Toc`

Other less commonly used components can be imported from `rspress/theme`, for example:

preview.mdx

```
import { SourceCode } from '@rspress/core/theme'

<SourceCode href="/" />
```

doom Components

`doom` provides some global components to assist in documentation writing, which can be used directly without import. Currently, these include:

Overview

Document overview component used to display the document directory

Directive

Sometimes, due to nested indentation, the [custom container](#) syntax may fail. You can use the `Directive` component as a replacement.

- The directory structure of multilingual documents (``doc/en``) needs to be exactly the same as the documents under the ``doc/zh`` directory to ensure that the links in multilingual documents are identical except for the language identifier.

```
<Directive type="danger" title="Note">
```

If you are using automated translation tools, you do not need to worry about

Show more ▾

- The directory structure of multilingual documents (`doc/en`) needs to be exactly the same as the documents under the `doc/zh` directory to ensure that the links in multilingual documents are identical except for the language identifier.

Note

If you are using automated translation tools, you do not need to worry about this issue. The automated translation tools will automatically generate the target language document directory structure based on `doc/zh`.

ExternalSite

Component for referencing external sites

```
<ExternalSite name="connectors" />
```

Note

Because DevOps Connectors releases on a different cadence from Alauda Container Platform, the DevOps Connectors documentation is now available as a separate documentation set at [DevOps Connectors](#)[↗].

ExternalSiteLink

Component for referencing external site links

```
<ExternalSiteLink name="connectors" href="link.mdx#hash" children="Content" />
```

[Content ↗](#)

TIP

In mdx, `<ExternalSiteLink name="connectors" href="link" children="Content" />` has a different meaning from the following content:

```
<ExternalSiteLink name="connectors" href="link">
  Content {/* will be rendered inside a `p` element */}
</ExternalSiteLink>
```

If you do not want the text to be rendered inside a `p` element, you can pass it using the `children` attribute as shown in the example above.

AcpApisOverview and ExternalApisOverview

Components for referencing external site API overviews

```
<AcpApisOverview />
{/* same as following */}
<ExternalApisOverview name="acp" />

<ExternalApisOverview name="connectors" />
```

Note

For the introduction to the usage methods of ACP APIs, please refer to [ACP APIs Guide ↗](#).

Note

For the introduction to the usage methods of DevOps Connectors APIs, please refer to [DevOps Connectors APIs Guide ↗](#).

Term

Term component, plain text, dynamically mounted and injected

```
<Term name="company" textCase="capitalize" />
<Term name="product" textCase="lower" />
<Term name="productShort" textCase="upper" />
<Term name="alaudaCloudLink" />
```

Alauda alauda container platform ACP [Alauda Cloud](#)

props

- `name` : Built-in term name, refer to [dynamic mount configuration file](#)
- `textCase` : Text case transformation, optional values are `lower` , `upper` , `capitalize`

TermsTable

Built-in term list display component

```
<TermsTable />
```

Name	Chinese	Chinese Bad Cases	English	English Bad Cases	Description
company	公司	-	Alauda	-	公司
product	容器平台	-	Alauda Container Platform	-	容器平台

Name	Chinese	Chinese Bad Cases	English	English Bad Cases	Description
productShort	ACP	-	ACP	-	□□□□□□
alaudaCloudLink	Alauda Cloud ↗	-	Alauda Cloud ↗	-	-

props

- `terms`: `NormalizedTermItem[]`, optional, custom term list for convenient reuse when rendering custom terms in internal documentation

JsonViewer

```
<JsonViewer value={{ key: 'value' }} />
```

yamljson

key: value

Custom Component Reuse

According to the [convention](#), we can extract reusable content into the `shared` directory, then import it where needed, for example:

```
import CommonContent from './shared/CommonContent.mdx'  
  
<CommonContent />
```

If you need to use more [runtime](#) related APIs, you can implement components with `.jsx/.tsx` and then import and use them in `.mdx` files.

```
// shared/CommonContent.tsx
export const CommonContent = () => {
  const { page } = usePageData()
  return <div>{page.title}</div>
}

// showcase/content.mdx
import { CommonContent } from './shared/CommonContent'
```

[Show more](#) ▾

WARNING

Note: Currently, components exported from `.mdx` do not support passing `props`. Refer to [this issue](#). Therefore, for scenarios requiring `props` passing, please develop using `.jsx/.tsx` components.

Internationalization

Most of the internal documentation of `alauda` is bilingual in Chinese and English. Therefore, we by default support using `en` / `zh` subfolders to store documents in different languages. It is recommended to also store static resources under `en` / `zh` subfolders in the `public` directory, which facilitates the management of both documentation content and static resources.

TOC

[i18n.json](#)[.ts/.tsx](#)[.mdx](#)

`i18n.json`

For reusable components, if you need to support both Chinese and English within the same component, you first need to create an `i18n.json` file under the `docs` directory. Then, in the component, use `useI18n` to get the text for the current language, for example:

```
docs/i18n.json
```

-

[Show more](#) ▾

.ts/.tsx

```
import { useI18n } from '@rspress/core/runtime'

export const CommonContent = () => {
  const t = useI18n()
  return <h1>{t('title')}</h1>
}
```

.mdx

```
import { useI18n } from '@rspress/core/runtime'

# {useI18n()('title')}

{useI18n()('description')}
```

API Documentation

According to actual business needs, we generally divide APIs into three types: standard K8S API, advanced API, and CRD (Custom Resource Definition). Therefore, the directory structure is usually organized as follows:

```
├─ apis
│   ├── advanced_api # Advanced APIs
│   ├── crds # CRDs
│   ├── kubernetes_api # K8S APIs
│   └─ references # Common references
```

TOC

K8S API

[props](#)

Advanced API

[props](#)

CRD (deprecated)

[props](#)

Common References

[props](#)

Specifying openapi Path

K8S API

```
kubernetes_apis/workload/daemonset.mdx
```

```
# DaemonSet [apps/v1]

<K8sAPI
  name="io.k8s.api.apps.v1.DaemonSet"
  pathPrefix="/kubernetes/{cluster}"
/>
```

Refer to [DaemonSet](#).

```
crds/ArtifactCleanupRun.mdx
```

```
# ArtifactCleanupRun

<K8sAPI name="artifactcleanupruns.artifacts.katanomi.dev" />
```

Refer to [ArtifactCleanupRun](#).

props

- `name`: Reference name under OpenAPI schema `definitions` (v2) or `components/schemas` (v3), or CRD `metadata.name`
- `namespaced`: Indicates whether the resource is namespace-scoped; defaults to `true`, meaning the API Endpoints include the namespace path parameter `namespaces/{namespace}`
- `pathPrefix`: Can be used to override the global configuration `api.pathPrefix`
- `filepath`: Similar to [specifying openapi path](#), used to specify a particular openapi or CRD file
- `apiGroup`: Optional, specifies the API group; openapi will try to read the referenced `x-kubernetes-group-version-kind`, same below
- `apiVersion`: Optional, specifies the API version; CRD will default to the first version in `spec.versions`

- `apiKind`: Optional, specifies the API resource kind

Advanced API

advanced_apis/codeQualityTaskSummary.mdx

```
# CodeQualityTaskSummary
```

```
<OpenAPIPath path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summ
```

Refer to [CodeQualityTaskSummary](#).

props

- `path`: Path under OpenAPI schema `paths`
- `pathPrefix`: Can be used to override the global configuration `api.pathPrefix`
- `openapiPath`: See [specifying openapi path](#)

CRD (deprecated)

WARNING

Please use the `K8sAPI` component instead of the `K8sCrd` component. The `K8sCrd` component will be removed in future versions.

crds/ArtifactCleanupRun-K8sCrd.mdx

```
# ArtifactCleanupRun - K8sCrd
```

```
<K8sCrd name="artifactcleanupruns.artifacts.katanomi.dev" />
```

Refer to [ArtifactCleanupRun-K8sCrd](#).

props

- `name` : CRD `metadata.name`
- `crdPath` : Similar to [specifying openapi path](#), used to specify a particular CRD file

Common References

```
references/CodeQuality.mdx
```

```
# CodeQuality
```

```
<OpenAPIRef schema="v1alpha1.CodeQuality" />
```

Refer to [CodeQuality](#).

props

- `schema` : Reference name under OpenAPI schema `definitions` (v2) or `components/schemas` (v3)
- `openapiPath` : See [specifying openapi path](#)

Specifying openapi Path

For the `OpenAPIPath` and `OpenAPIRef` components, by default, all openapi definition files are searched until a match is found. If you need to specify a particular openapi file, you can use the `openapiPath` attribute:

```
<OpenAPIPath
  path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summary"
  openapiPath="shared/openapis/katanomi.json"
/>
```

Permission Description Document

```
<K8sPermissionTable functions={['devops-testplans', 'devops-testmodules']} />
```

TOC

[props](#)[Example](#)

props

- `functions`: `string[]` - Required. An array of `FunctionResource` resource names to be displayed.

Example

Function	Action	Platform Administrator	Platform auditors	Project Manager	Namespace Administrator
testplans	View	✓	✓	✓	✓
devops-testplans	Create	✓	✗	✓	✓

Function	Action	Platform Administrator	Platform auditors	Project Manager	Namespace Administrator
testmodules devops-testmodules	Update	✓	×	✓	✓
	Delete	✓	×	✓	✓
	View	✓	✓	✓	✓
	Create	✓	×	✓	✓
	Update	✓	×	✓	✓
	Delete	✓	×	✓	✓

Referencing Documents

In Markdown files:

```
<!-- reference-start#name -->

<!-- reference-end -->
```

In MDX files:

```
{/* reference-start#name */}

{/* reference-end */}
```

The `name` above refers to the name of the referenced document. For more information, please refer to [Document Reference Configuration](#). If the referenced document content uses static resources from a remote repository, the related static resources will be automatically stored locally in the `<root>/public/_remotes/<name>` directory.

Here is an example using `<!-- reference-start#ref -->`:

TOC

Document Reference Configuration

`frontmatterMode`

Document Reference Configuration

reference:

- **repo:** `alauda-public/product-doc-guide` # Optional, repository address for the referenced document. If not provided, the current document repository address will be used by default.

branch: # [string] Optional, branch of the referenced document repository.

publicBase: # [string] Optional, the directory where static resources for remote repository located, corresponding to absolute paths like `/imag`

Show more ▾

frontmatterMode

- **ignore**: Ignores the frontmatter of the referenced document and retains the frontmatter of the current document.
- **merge**: Merges the frontmatter of the referenced document. If there are the same keys, the values from the referenced document will overwrite those in the current document.
- **replace**: Replaces the frontmatter of the current document with that of the referenced document.
- **remove**: Removes the frontmatter of the current document.

For writing documentation, refer to [Document Reference](#).

Deployment

TOC

Build and Preview

Multi-version Build

Merged Directory Structure

Dynamic Mount Configuration File

Build and Preview

Before deployment, we need to build the project for production and preview it locally to ensure the project runs correctly:

```
doom build # Build static assets
doom serve # Preview the build assets in production mode
```

Multi-version Build

By default, `doom build` outputs the build artifacts to the `dist` directory. If you need to build multiple versions of the documentation, you can specify the version number with the `-v` parameter, for example:

```
# Usually determined by branch name, e.g., release-4.0 corresponds to version 4.0
doom build -v 4.0 # Build version 4.0, output to dist/4.0, documentation access path is {base}/4.0
doom build -v master # Build master version, output to dist/master, documentation access path is {base}/master
doom build -v {other} # Build other versions, output to dist/{other}, documentation access path is {base}/{other}
```

[Show more](#) ▾

Merged Directory Structure

```
|— console-platform
|   |— 4.0
|   |— 4.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
|— console-devops-docs
|   |— 4.0
```

[Show more](#) ▾

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Redirecting...</title>
    <meta http-equiv="refresh" content="0; url=/console-docs/4.1" />
  </head>
```

[Show more](#) ▾

Dynamic Mount Configuration File

overrides.yaml

```
# Documentation information, each document can mount to override default conf
title:
  en: Doom - Alauda
  zh: Doom - 喵喵
logoText:
  en: Doom - Alauda
```

[Show more](#) ▾

versions.yaml

```
- '4.1'
- '4.0'
```